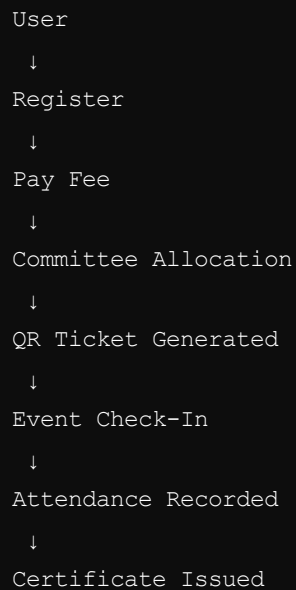


If you want this to look like a real production application rather than a college project, design it around **business workflows**, not around pages.

Your PDF defines the business flow as:



```
graph TD; User --> Register; Register --> Pay_Fee[Pay Fee]; Pay_Fee --> Committee_Allocation[Committee Allocation]; Committee_Allocation --> QR_Ticket_Generated[QR Ticket Generated]; QR_Ticket_Generated --> Event_Check_In[Event Check-In]; Event_Check_In --> Attendance_Recorded[Attendance Recorded]; Attendance_Recorded --> Certificate_Issued[Certificate Issued];
```

Everything should be built around this lifecycle.

 433ac050-72bd-4a57-b9b3-c961489...

1. Tech Stack

Frontend

- React + Vite
- TypeScript
- TailwindCSS
- React Query
- React Hook Form
- Zod

Backend

- Node.js
- Express.js
- TypeScript

Database

- MongoDB Atlas

File Storage

- Cloudinary or AWS S3

Store:

- Certificates
- Event banners
- Committee assets

Never store PDFs inside MongoDB.

Authentication

- JWT Access Token
- Refresh Token
- HttpOnly Cookies

Payments

- Razorpay

Deployment

Frontend:

- [Vercel ↗](#)

Backend:

- [Render ↗](#) or [Railway ↗](#)

Database:

- [MongoDB Atlas ↗](#)

2. Folder Structure

```
src/
```

```
├─ config
├─ middleware
├─ controllers
├─ services
├─ repositories
├─ routes
├─ models
├─ utils
├─ validators
├─ jobs
├─ types
└─ app.ts
```

Separation:

Controllers

Receive request.

```
req
↓
controller
```



Services

Business logic.

```
allocateCommittee()
issueCertificate()
verifyPayment()
```



Repository

Database operations.

```
User.find()
Registration.create()
```



3. Database Design

Users

</> JavaScript



```
{
  _id,

  name,
  email,

  passwordHash,

  role: "participant" | "organizer" | "admin",

  isVerified,

  createdAt
}
```

Events

JavaScript



```
{
  _id,

  title,
  description,

  venue,

  startDate,
  endDate,

  registrationDeadline,

  status,

  bannerUrl
}
```

Committees

JavaScript



```
{
  _id,

  eventId,

  name,

  agenda,

  capacity,

  filledSeats
}
```

Registrations

JavaScript



```
{
  _id,
```

```
    userId,  
  
    eventId,  
  
    committeeId,  
  
    status,  
  
    paymentStatus,  
  
    allocatedAt,  
  
    attendanceStatus  
  }  
}
```

Payments

JavaScript



```
{  
  _id,  
  
  registrationId,  
  
  razorpayOrderId,  
  
  razorpayPaymentId,  
  
  amount,  
  
  currency,  
  
  status,  
  
  verifiedAt  
}
```

Tickets

JavaScript



```
{  
  _id,  
  
  registrationId,  
  
  eventId,  
  
  committeeId,  
  
  status,  
  
  paymentStatus,  
  
  allocatedAt,  
  
  attendanceStatus  
}
```

```
qrToken,  
  
issuedAt  
}
```

Certificates

</> JavaScript



```
{  
  _id,  
  
  registrationId,  
  
  certificateUrl,  
  
  issuedAt  
}
```

4. Authentication Flow

Register

POST /auth/register



User creates account.

Password:

```
password  
↓  
bcrypt  
↓  
hash stored
```



Never store plain text.

Login

POST /auth/login



Return:

```
Access Token
Refresh Token
```



Store:

```
Refresh Token → HttpOnly Cookie
```



Refresh

```
POST /auth/refresh
```



Generates new access token.

Logout

```
POST /auth/logout
```



Invalidate refresh token.

5. Route Design

Auth

```
POST /api/auth/register
```



```
POST /api/auth/login
```

```
POST /api/auth/refresh
```

```
POST /api/auth/logout
```

Events

```
GET /api/events
```



```
GET /api/events/:id

POST /api/events

PATCH /api/events/:id

DELETE /api/events/:id
```

Admin only:

```
POST
PATCH
DELETE
```



Committees

```
GET /api/events/:eventId/committees

POST /api/events/:eventId/committees

PATCH /api/committees/:id

DELETE /api/committees/:id
```



Registration

```
POST /api/registrations

GET /api/registrations/me

GET /api/registrations/:id
```



Payment

```
POST /api/payments/create-order

POST /api/payments/verify

POST /api/payments/webhook
```



Webhook is important.

Do not rely only on frontend callbacks.

Ticket

```
GET /api/tickets/:registrationId
```



```
POST /api/tickets/generate
```

Attendance

```
POST /api/attendance/checkin
```



```
POST /api/attendance/checkout
```

```
GET /api/attendance/report
```

Certificate

```
POST /api/certificates/generate
```



```
GET /api/certificates/:id
```

```
GET /api/certificates/download/:id
```

6. RBAC (Role-Based Access Control)

Roles:

```
Participant
```



```
Organizer
```

```
Admin
```

Middleware:

```
<> TypeScript
```



```
authorize(["admin"])
```

Example:

```
Participant
❌ create event

Organizer
❌ delete event

Admin
✅ everything
```



7. Razorpay Flow

Proper production flow:

```
User clicks Pay
↓
Backend creates Razorpay Order
↓
Frontend opens Razorpay Checkout
↓
User pays
↓
Razorpay returns payment data
↓
Backend verifies signature
↓
Payment marked successful
↓
Registration activated
```



Never:

```
Frontend says success
↓
Trust frontend
```



That's insecure.

8. Committee Allocation Logic

Critical area.

Imagine:

```
Committee Capacity = 50
```

```
Filled = 49
```

```
Two users pay together
```



Without atomic updates:

```
User A gets seat
```

```
User B gets seat
```

```
Filled = 51
```



Bug.

Use Mongo transactions.

Pseudo:

```
⟨⟩ TypeScript
```

```
startSession()
```

```
check capacity
```

```
increment filledSeats
```

```
create allocation
```

```
commit
```



9. QR Architecture

QR should NOT contain:

```
Name
```

```
Email
```

```
Phone
```



Instead:

```
⟨⟩ JSON
```



```
{  
  "ticketId": "abc123"  
}
```

At gate:

```
Scan  
↓  
Backend Validation  
↓  
Mark Attendance
```



10. Certificate Architecture

After attendance:

```
Attendance = Present  
↓  
Generate PDF  
↓  
Upload S3  
↓  
Store URL  
↓  
Email User
```



Certificate should be generated once.

Not every download.

11. Security Features

Helmet

```
</> TypeScript  
  
app.use(helmet())
```



Security headers.

Rate Limiting

⌄ TypeScript



```
5 login attempts/minute
```

⌄ TypeScript



```
express-rate-limit
```

CORS

Allow only:

yourdomain.com



Not:

*



Input Validation

Use:

⌄ TypeScript



```
zod
```

Every endpoint.

Mongo Sanitization

Prevent:

⌄ JavaScript



```
{  
  "$gt": ""  
}
```

attacks.

Use:

```
express-mongo-sanitize
```



XSS Protection

Sanitize rich text.

Especially:

```
Event Description  
Committee Description
```



CSRF Protection

Needed if using cookies.

Use:

```
csrf
```



Audit Logs

Create collection:

```
</> JavaScript
```



```
{  
  userId,  
  
  action,  
  
  timestamp  
}
```

Track:

```
Payment Verified  
  
Certificate Issued  
  
Attendance Marked
```



12. Scaling to 1,000+ Users

At 1000 users:

No microservices needed.

Architecture:

React (Vercel)

↓

Express API (Render)

↓

MongoDB Atlas

↓

Cloudinary/S3

↓

Razorpay



This can comfortably support:

- 1,000–5,000 users
- 100–300 concurrent users
- Thousands of registrations

13. Resume-Worthy Enhancements

Add:

Email Service

Use:

- [Resend ↗](#)
- [SendGrid ↗](#)

For:

- Registration confirmation
- Payment success
- Certificate issued

Background Jobs

Use:

```
BullMQ  
Redis
```



Jobs:

```
Generate Certificate  
  
Send Email  
  
Generate QR
```



Monitoring

Use:

- [Sentry ↗](#)

Track:

```
API crashes  
Payment failures  
Unhandled exceptions
```



If you build the architecture above with proper RBAC, payment verification, transactions for committee allocation, QR-based attendance, certificate generation, audit logs, and deployment, it will look much closer to a startup-grade event platform than a typical internship project.